



European
Power Platform
Conference

WELCOME
TO

COPEN
HAGEN 20
26



JOIN THE
CONVERSATION
#EPPC26



Multi-Agent Systems

with Microsoft Copilot Studio, A2A and MCP

Jennifer Krause

Consultant — Agentic Systems & Automation
NTT DATA Inc.

Holger Imbery

Principal Architect — Agentic Systems
NTT DATA Inc.

Multi-Agent Systems

with Microsoft Copilot Studio, MCP & A2A



Jennifer Krause

Consultant — Agentic Systems & Automation
NTT DATA Inc.



Holger Imbery

Principal Architect — Agentic Systems
NTT DATA Inc.



WHY THIS SESSION

We don't evangelize agents.

We ship them for enterprises,
and we've watched some of these patterns break.

Here's what we learned,
so that yours don't.

Real engagements · GA features · honest about what's in preview

PLEASE NOTE

Scope of this session.

We commit to features available **today**. The future roadmap is clearly marked as **in preview**.

IN SCOPE — GENERALLY AVAILABLE

The building blocks, patterns and protocols shipped today.

- Child and connected agents, generative orchestration
- MCP (Streamable transport) and A2A — both GA
- Autonomous triggers, Dataverse state, Purview audit
- Configuration paths and UI as they exist now

OUTLOOK ONLY — PREVIEW

The closing outlook *looks ahead* — *not* a production commitment.

- The Build 2026 redesign (Skills, Memory)
- New orchestrator and workflow designer
- Public preview — may change before GA
- Shown for direction, clearly labelled preview

AGENDA

What we'll cover.

From the why, through patterns and protocols, to governance — with three short demos and a look ahead.

01

The why

Why go multi-agent — and when not to.

02

Building blocks

Child and connected agents — two relationships.

03

Patterns & scenarios

Three shapes; four real-world blueprints.

04

Protocols

MCP for tools. A2A for agent-to-agent.

05

In practice & outlook

Governance, adoption — and a preview of what's next.

THE WHY

Why multi-agent inside Copilot Studio.

Single-agent designs hit a wall — tool-selection accuracy degrades past roughly 30–40 actions. Specialization, governance and reuse demand a multi-agent model.

Specialization

Each agent owns a narrow domain, a clear tool set, and a bounded prompt — less drift, better accuracy.

Reuse

Package an agent once, use it from many orchestrators — instead of re-implementing logic in every agent.

Separation of concerns

Different owners, different knowledge sources, different guardrails — without one monolithic agent.

Scale & governance

Version, publish and audit each agent on its own lifecycle; identity and policy apply per agent.

Multi-agent is the answer to accuracy, reuse and ownership — not to complexity.

PART 01

Building blocks.

Two agent relationships in Copilot Studio.

Child agents — inside your orchestrator.
Connected agents — reached over an open protocol.

Same platform. Two very different relationships.

THE MODEL

Two ways agents work together.

Same orchestration in Copilot Studio. Two very different relationships between agents.

CHILD AGENTS

Composition inside one orchestrator.

- Built within your main agent in Copilot Studio, invoked as a tool
- Logically group tools, instructions or knowledge within a main agent
- Shared governance, configuration settings and lifecycle
- Natural fit for sub-tasks in one domain

CONNECTED AGENTS

Interop across agent boundaries.

- Added on the Agents tab (CS agent, or external via A2A)
- Can live in another team, tenant or vendor
- Own lifecycle, own identity, own policies
- Natural fit for cross-domain capabilities

CHILD AGENTS

Composition inside your orchestrator.

A parent agent delegates a sub-task to a published agent in the same environment.

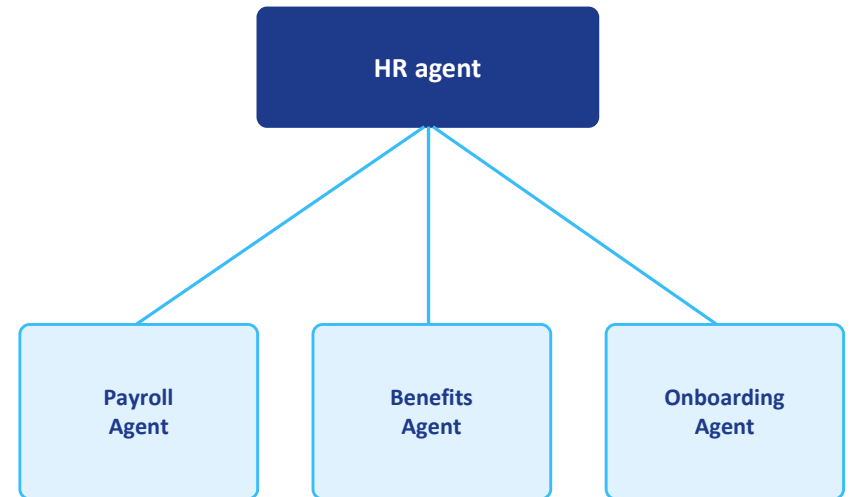
WHAT IT IS

A Copilot Studio agent called as a tool by another. It has its own set of instructions and optional input/output variables; reference it from instructions with /. The parent orchestrates; the child executes a focused task.

WHEN TO USE

- Sub-tasks inside one business domain
- Same data boundary and governance owner
- Reuse across a small set of related agents
- You want one audit trail end-to-end

PARENT → CHILD



One environment · shared knowledge · shared policy

CONNECTED AGENTS

Interop across agent boundaries.

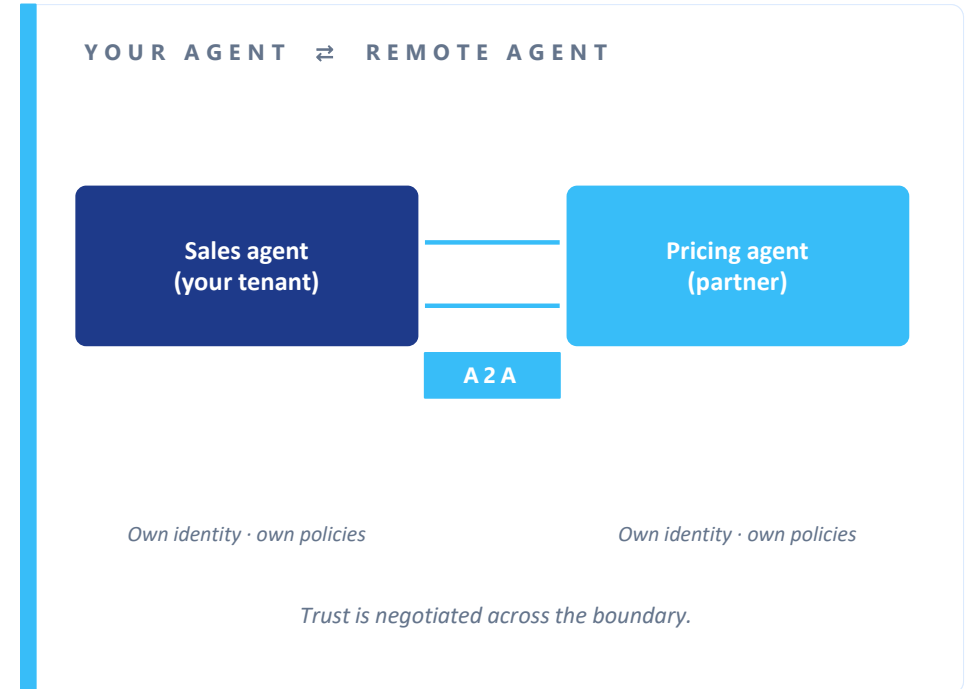
Your agent reaches another agent that lives outside your own build — over an open protocol.

WHAT IT IS

A separate agent that your Copilot Studio agent calls through an agent-to-agent protocol. Two independent agents negotiate a task.

WHEN TO USE

- Capability lives in another team or tenant
- Third-party or partner agent
- Different lifecycle, identity or policy owner
- Cross-domain workflow spanning systems



DECISION GUIDE

Child or connected? Pick by boundary.

The decision is not about size — it's about who owns the agent, its data, and its lifecycle.

Dimension	Child agent	Connected agent
Built in	Within your Agent – in Copilot Studio, same environment	Any A2A-speaking platform
Ownership	Same team / business owner	Different team, tenant or vendor
Identity & policy	Shared	Separate, negotiated
Lifecycle	Versioned together	Independent release cycle
Typical scope	Sub-task in one domain	Cross-domain or partner capability
Example	HR agent → Payroll child	Sales agent → Pricing agent

Rule of thumb: same boundary → child. Crosses a boundary → connected.

EXAMPLE · CHILD AGENT

HR agent with a payroll child.

One business domain, one governance owner — composition inside a single orchestrator.

SCENARIO

An employee asks the HR agent: "When is my next pay date and what is my gross?" The HR agent delegates the payroll look-up to a child agent that owns the HCM (Human Capital Management) connection.

1

User asks HR

Natural-language question in Microsoft 365 Chat.

2

HR agent routes

Orchestrator recognises a payroll intent.

3

Child executes

Payroll child agent calls the HCM system and returns data.

4

HR agent replies

Composes the answer in the employee's language.

Same tenant · shared identity · one audit trail.

EXAMPLE · CONNECTED AGENT

Sales agent talking to a partner pricing agent.

Two independently-owned agents negotiate a cross-domain task over A2A.

SCENARIO

A seller asks: "Give me a quote for this configuration in the partner catalogue." The Sales agent reaches a partner-operated pricing agent via A2A to produce a validated quote.

YOUR TENANT

Sales agent

Entra identity · Purview labels
your guardrails

A2A

PARTNER TENANT

Pricing agent

Partner identity
partner data & policies

Each agent keeps its own identity, data and policy. The protocol carries the task — not the trust boundary.

DEMO

Child & connected agents in Copilot Studio.

Add a child agent — its own instructions, inputs and outputs

Add a connected Copilot Studio agent from the environment

Run a prompt — watch the orchestrator delegate in the activity map

Copilot Studio

Environment dev-na-agents

Holger Imbery

HI

Home

AgentOps

Agents

Flows

Tools

...

Solution Architect

Overview Knowledge Tools Agents Topics Activity Evaluation Analytics Channels

Published 4/13/2026

Publish

Settings

...

Test

Details

Name

Solution Architect

Description

354/1024

Top-level orchestrator agent that analyzes user requests and delegates tasks to specialized child agents. It acts as an AI solutions architect, determining which sub-agent (architecture design, extension, configuration, cost, advanced integration, content, design prompt, or diagram) is best suited to handle each query, then compiles the final response.

Agent status

1 Warning

Review

Select your agent's model

Your agent will primarily use the model for reasoning and responding. Experimental models are subject to [preview terms](#). [Learn more](#)

Claude Sonnet 4.6

Analytics

Open Analytics

Check your agent's key performance info from the last 7 days.

Conversation sessions	Total reactions	Engagement
0 0%	0 0%	0% 0%

Instructions

Edit

You are the Solution Architect, an AI agent that coordinates with multiple specialized sub-agents to design and implement agentic systems on the Microsoft platform.

When a user request is received, analyze the intent and determine which of your child agents can best fulfill the task:

- If the query involves architecture design, multi-agent workflows, writing agent instructions, or designing Dynamics 365 Customer Service / Contact Center agents, delegate to Architecture & Scenario Des...
- If the query is about extending or customizing Microsoft 365 Copilot, Dynamics 365 Copilot, or other Copilot implementations, use the Copilot Extension Advisor

PART 02

Architectural patterns.

Three topology shapes — plus four scenario blueprints from real builds.

Orchestrator · Specialist network · Hub-and-spoke

Pick the shape that matches ownership, weight, and flow.

PATTERN 01

Orchestrator with specialists.

One main agent owns the conversation. Everything else is a tool it can call.

SHAPE

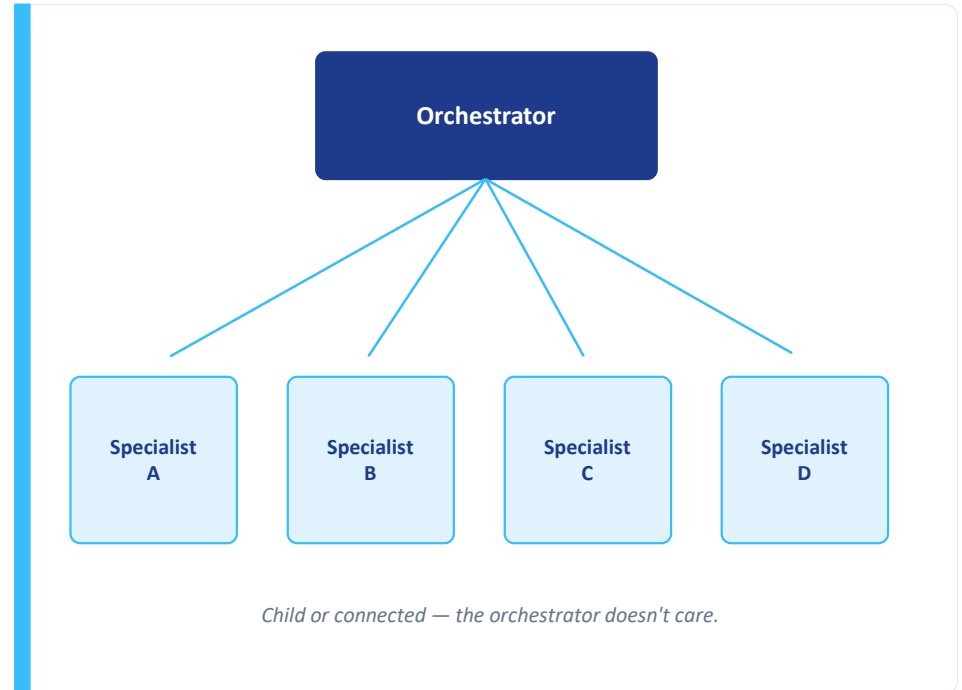
A single orchestrator agent interprets intent, plans steps, and dispatches to child or connected agents. The user only ever sees the orchestrator.

WHEN IT FITS

- One dominant business use case
- Clear primary owner of the experience
- Most calls stay inside one domain

WATCH OUT

The orchestrator can become a bottleneck as specialists grow. Keep its prompt narrow — it decides; it doesn't execute.



PATTERN 02

Specialist network.

Peer agents that hand off to each other when a task crosses a domain boundary.

SHAPE

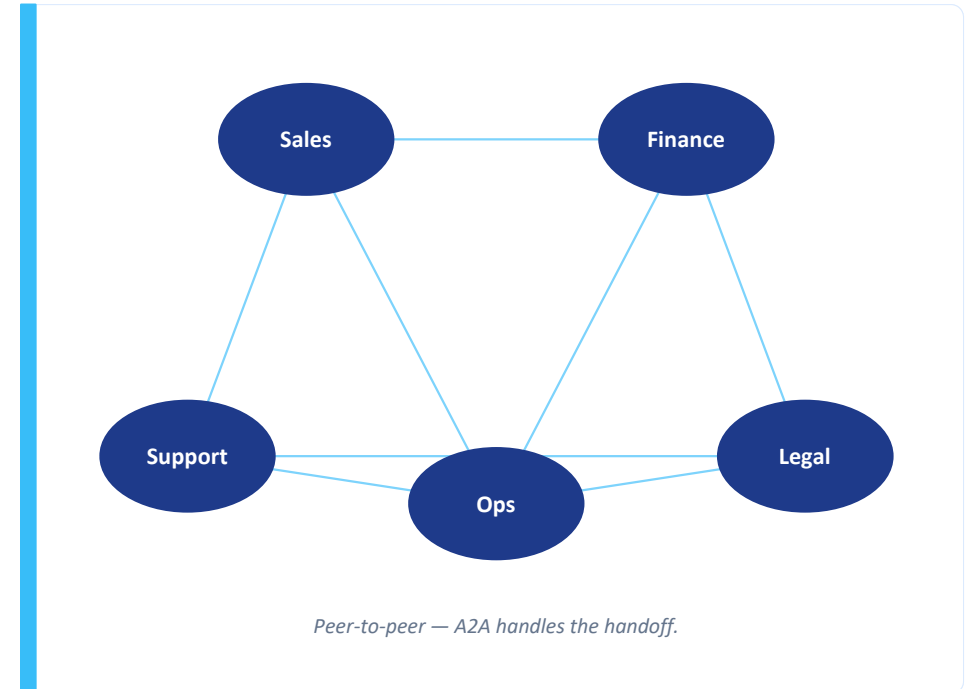
No single orchestrator. Each agent is a domain expert and knows which peers to invoke for tasks outside its domain — typically over A2A.

WHEN IT FITS

- Multiple domains, comparable weight
- Strong independent ownership per agent
- Workflows often cross domain lines

WATCH OUT

Harder to reason about. Publish clear agent cards and keep the number of peers small.



PATTERN 03

Hub-and-spoke.

A central agent mediates access to shared capabilities used by many front-end agents.

SHAPE

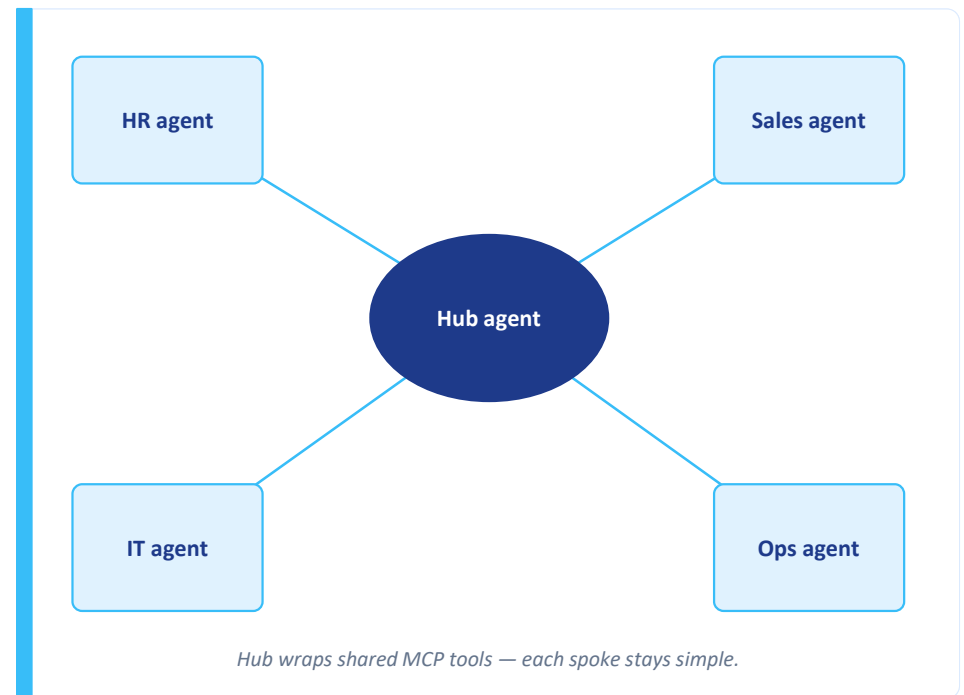
Front-end agents serve distinct user groups. A shared hub agent owns cross-cutting tools — identity, retrieval, compliance checks — exposed via **MCP**.

WHEN IT FITS

- Multiple agents, one shared back-end
- Cross-cutting governance concerns
- You want to avoid tool sprawl

WATCH OUT

The hub is a single point of dependency. Design it to fail gracefully and measure latency carefully.



Four scenarios

Real-world blueprints, each on a different orchestration pattern — all buildable on GA features.

IT incident triage

Router-worker. Classify the ticket, delegate to a runbook specialist and a ticket writer. Structured outputs between agents.

Contract review

Parallel fan-out. Legal, finance and risk reviewers run at once over scoped policy libraries; an orchestrator merges the memo.

Field-service dispatch

Autonomous trigger. A Dataverse row wakes the agent; it checks stock, reschedules and notifies — no user in the loop.

Employee onboarding

Hierarchical. A supervisor coordinates HR, IT and facilities specialists; state lives in a Dataverse table, with SLA escalation.

Onboarding: state is the architecture.

Long-running, cross-department work. The Dataverse state table — not the conversation — is the source of truth.

State store

One OnboardingCase row per hire: each step's status, timestamps and links. Conversation history truncates; Dataverse does not.

Supervisor stays small

It reads state and delegates — never executes. Short instructions, explicit order, some parallelism (IT and facilities at once).

Specialists are boring

Each does one thing and writes only its own columns. Cleverness in the wrong place is a reliability problem.

SLA monitoring

The supervisor escalates overdue steps to a Teams channel. Without the SLA rule, a stuck step looks fine until it isn't.

Protocols.

Two open standards carry the weight of multi-agent interop.

MCP — tools for agents.

A2A — agents talking to agents.

Complementary, not alternatives.

MCP · MODEL CONTEXT PROTOCOL

One protocol between agents and tools.

MCP gives agents a standard, discoverable way to reach tools, data sources and prompts.

Tools

Functions the agent can invoke — search, create, update, retrieve. Described in a schema.

Resources

Data and documents an agent can read on demand — files, records, knowledge items.

Prompts

Reusable prompt templates the server exposes, so agents use consistent instructions.

WHY IT MATTERS

Before MCP, every agent wired every tool by hand. With MCP, any compliant server exposes its tools the same way — and any compliant agent can consume them.

Discover once. Reuse everywhere. Govern centrally.

How to use MCP in Copilot Studio.

Practical rules that pay off at scale.

- | | | |
|----|---|--|
| 01 | Treat the MCP server as the tool contract. | Keep it versioned and documented. Agents depend on the contract, not the implementation. |
| 02 | One domain per server. | Group related tools behind one server. Avoid a “god”-server — it becomes hard to govern and test. |
| 03 | Map identity carefully. | Decide under which identity the tool is called — agent, user, or a workload. Enforce in Entra. |
| 04 | Surface tool schemas clearly. | Clear names and descriptions let the orchestrator pick the right tool on the first try. |
| 05 | Observe every call. | Log inputs, outputs, latencies and failures. Treat MCP traffic like any other critical dependency. |

DEMO

Connect an MCP server to an agent.

Tools → Add a tool → New tool → Model Context Protocol

Enter the server URL and description; choose OAuth 2.0

Tools auto-discovered → agent calls one → structured result

Home

AgentOps

Agents

Flows

Tools

Copilot Studio

Environment
dev-na-agents

?

Holger Imbery

HI

Solution Architect

Overview

Knowledge

Tools

Agents

Topics

Activity

Evaluation

Analytics

Channels

Published 4/13/2026

Publish

Settings

Test

+ Add a tool

Search tools

All

Connector (1)

Model Context Protocol (5)

Last refreshed now

Name	Type	Available to	Trigger	Last modified	Errors	Blocked	Enabled
Request for information	Connector	Solution Architect	By agent	Holger Imbery 2 month...			☒ On
Microsoft Learn Docs MCP Server	Model Context Protocol	Solution Architect	By agent	Holger Imbery 2 month...			☒ On
Work IQ Calendar (Preview)	Model Context Protocol	Solution Architect	By agent	Holger Imbery 2 month...			☒ On
Work IQ Copilot (Preview)	Model Context Protocol	Solution Architect	By agent	Holger Imbery 2 month...			☒ On
Work IQ User (Preview)	Model Context Protocol	Solution Architect	By agent	Holger Imbery 2 month...			☒ On
Work IQ Word (Preview)	Model Context Protocol	Solution Architect	By agent	Holger Imbery 2 month...			☒ On

One protocol between agents.

Where MCP connects an agent to its tools, A2A connects an agent to another agent.

THE CORE IDEA

Each agent communicates an agent card that describes what it can do and how to talk to it. Another agent reads the card, negotiates a task, and exchanges messages — without sharing its internals.

KEY CONCEPTS

- Agent card — capabilities, endpoints, auth
- Tasks — units of work with state and messages
- Messages & artifacts — the payloads that flow
- Open, framework-agnostic — works beyond Microsoft

USE A2A WHEN

- Agent lives outside your build
- You need to cross tenant or vendor
- Independent lifecycle required
- Task exchange, not data pipe

DESIGN RULES

- Publish a minimal, clear agent card
- Negotiate auth at the boundary
- Keep tasks idempotent and resumable
- Log every exchange end-to-end

DEMO

End-to-end: orchestrator to a partner agent over A2A.

Agents tab → Add an agent → Connect to an external agent → Agent2Agent

Endpoint URL + auth; the agent card fills name & description

Delegate a task across the boundary → structured response + audit trail

Home

AgentOps

Agents

Flows

Tools

...

Copilot Studio

Environment
dev-na-agents

?

Holger Imbery

Hi

Solution Architect

Overview

Knowledge

Tools

Agents

Topics

Activity

Evaluation

Analytics

Channels

Published 4/13/2026

Publish

Settings

...

Test

+ Add an agent

Search

Last refreshed now

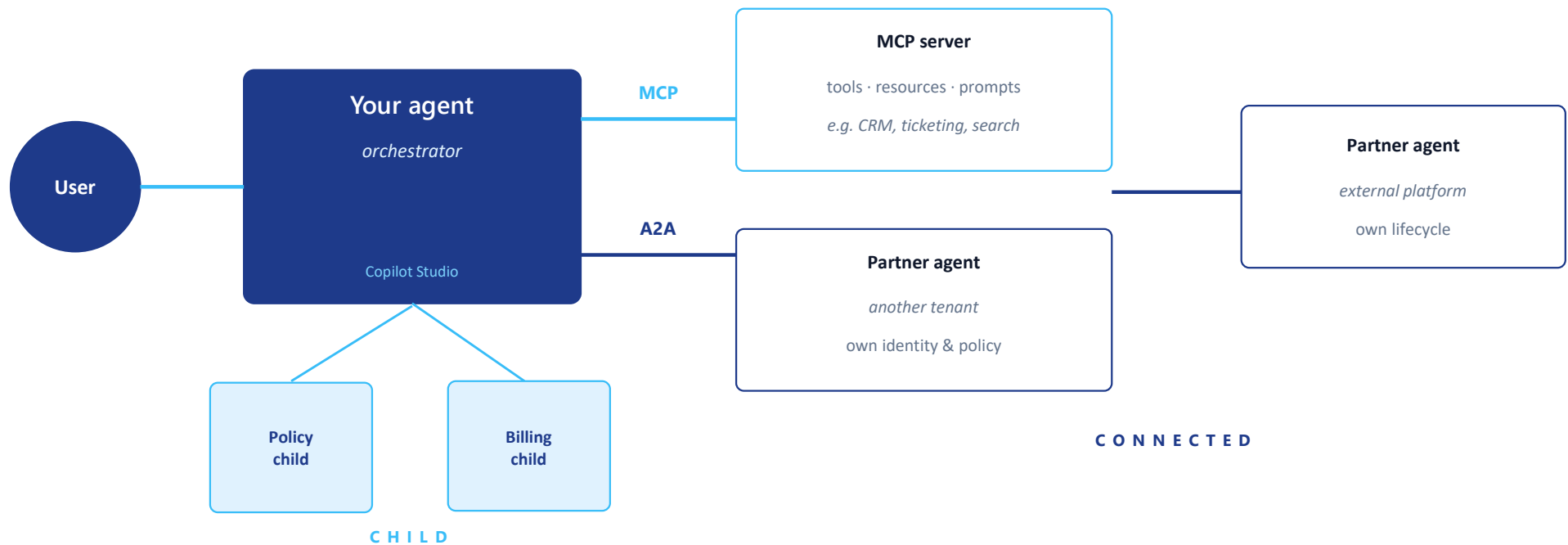
Name	Relationship	Trigger	Last modified	Errors	Blocked	Enabled
Advanced Integration Specialist	Child	By agent	Holger Imbery 2 months ago			On
Architecture & Scenario Designer	Child	By agent	Holger Imbery 2 months ago			On
Blog Article Writer	Child	By agent	Holger Imbery 2 months ago			On
Configuration Expert	Child	By agent	Holger Imbery 2 months ago			On
Content Creator	Child	By agent	Holger Imbery 2 months ago			On
Copilot Extension Advisor	Child	By agent	Holger Imbery 2 months ago			On
Design Prompt Generator	Child	By agent	Holger Imbery 2 months ago			On
Diagram Maker	Child	By agent	Holger Imbery 2 months ago			On
Licensing & Cost Advisor	Child	By agent	Holger Imbery 2 months ago			On
professional services pricing agent	Connected	By agent	Holger Imbery 11 minutes ago			On

This AI-powered feature is currently in preview. [See terms](#)

IN PRACTICE

How it all fits together.

Child agents compose, connected agents interoperate, MCP carries tools, A2A carries tasks.



MCP handles tool access inside the boundary. A2A handles agent collaboration across it.

PART 04

In practice.

Govern it, ship it, and measure what matters.

Governance & security, per agent

The anti-patterns that bite first

Start with a thin slice

Govern and secure every agent.

Identity, least privilege and audit — designed in, not bolted on.

- | | | |
|----|--|--|
| 01 | One identity per agent. | Use Entra Agent ID so permissions are scoped per agent, not per maker. An IT triage agent never needs HR write access. |
| 02 | Least privilege by default. | Read-only where reading is enough; write scopes only where a tool writes. Separate read and write servers. |
| 03 | Treat inputs as untrusted. | Trigger payloads and retrieved content can carry injected instructions (e.g. CVE-2026-21520). Validate; never let free text override instructions. |
| 04 | Structured, idempotent contracts. | Agents exchange typed data, not prose. Every write takes an idempotency key so retries don't duplicate tickets or emails. |
| 05 | Audit and observe. | Log every call and decision to Purview; apply DLP to the connectors behind your tools. Treat agents like products with owners. |

What bites first — and the fix.

Five failure modes seen in real Copilot Studio builds.

- | | | |
|----|--------------------------|---|
| 01 | The god agent. | Fifty tools, nine knowledge sources, an 8,000-character prompt. Fix: split by domain at ~30–40 actions. One agent, one domain, one owner. |
| 02 | Prose contracts. | Parsing ‘I created INC-12345’ breaks the day the wording changes. Fix: structured in, structured out — prose is for users, not agents. |
| 03 | Silent retries. | A timed-out call that actually succeeded creates duplicates. Fix: an idempotency key on every write. |
| 04 | No state store. | Relying on conversation history for multi-step work loses the thread. Fix: persist state in Dataverse. |
| 05 | Measuring volume. | ‘10,000 questions answered’ is a count, not an outcome. Fix: measure resolution, correctness and time-to-resolve. |

IN PRACTICE · ADOPTION

Start with a thin slice.

Ship one end-to-end path before you scale the graph of agents.

SCENARIO

Pick one real use case with a clear owner. Build the smallest version that delivers value end-to-end, then iterate — resist starting with a specialist network.

1

One orchestrator + one child

A narrow front door and a single focused specialist, in one environment.

2

Add one MCP server

Wrap one backend (start read-only). Prove the integration pattern.

3

Add one connected agent

Bring in a cross-boundary capability over A2A when you genuinely need it.

4

Measure outcomes

Track resolution and correctness, assign an owner, then expand.

Thin slice first · measure · then scale.

Outlook: the next Copilot Studio.

Announced at Build 2026 — public preview, subject to change. Direction, not commitment.

A streamlined four-surface agent model

A new orchestrator and workflow designer

Preview — verify status before you rely on it

PREVIEW · WHAT'S CHANGING

What's changing with the Build 2026 Announcement?

A rebuilt Copilot Studio, in public preview today and coexisting with the classic experience. Subject to change.

Four surfaces

Nine configuration tabs collapse to four — Skills, Tools, Knowledge, Connected agents — plus a Memory surface in preview.

New orchestrator

A coding-harness / CLI engine with stronger instruction-following, long-horizon and recursive tasks, and rich file outputs.

New workflow designer

A unified canvas with agent nodes that drop existing agents into a flow, plus MCP servers inside workflows (preview).

Foundry convergence

One runtime, two front doors. Copilot Studio and Foundry share A2A and the Microsoft IQ knowledge layer.

PREVIEW · SKILLS & MEMORY

Skills, Memory — and how to prepare.

The new model factors behaviour differently — and you can get ready today.

Skills

Reusable behavior authored as load-on-demand markdown. Import existing GitHub Copilot or Claude Code skills directly. Can combined with executable code

Memory

Persistent cross-session context via Work IQ — its Data, Memory and Inference layers ground answers in your work.

Child agents, folded in

In-agent complexity becomes Skills; cross-agent work stays Connected agents. The three-way split simplifies.

HOW TO PREPARE

Keep “classic” (GA) in production. Pilot the new experience in a non-production environment. Keep each piece of behaviour small and single-purpose, so it maps cleanly onto a Skill later. Re-check status before you commit.

Preview today. Coexists with classic. Verify before you rely.

TAKEAWAYS for today

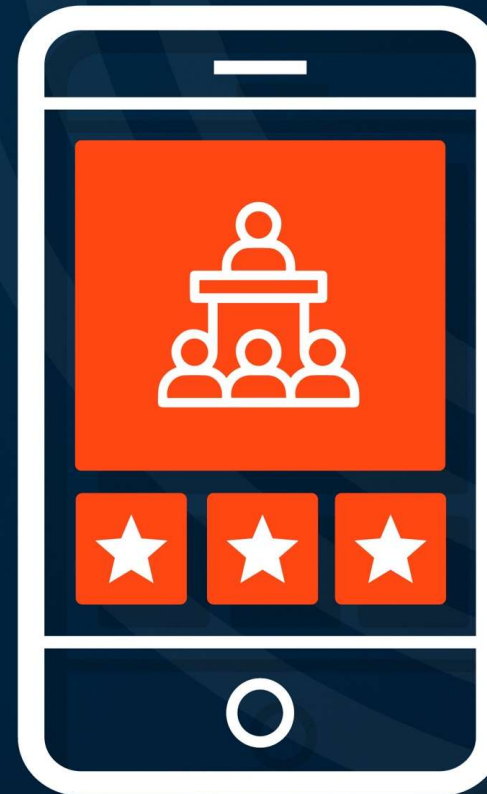
Principles to adopt.

Design the boundary first — the pattern and the protocol follow.

- | | | |
|----|--|---|
| 01 | Same boundary → child. Crosses a boundary → connected. | <i>Ownership decides the shape — not size.</i> |
| 02 | Keep the orchestrator instructions narrow. | <i>It decides. Specialists execute. Don't let it grow a domain.</i> |
| 03 | Use MCP for tools, A2A for agents. | <i>They are complementary — not alternatives. Adopt both.</i> |
| 04 | Version and observe every agent. | <i>Treat each agent like a product: cards, versions, telemetry, owners.</i> |



Please rate
this session
on the app



cvent

